

Documentación técnica

Terbol Ecommerce WebApp

Proyecto	terbol-webapp
Versión	0.1.0
Stack	Next.js 16 - React 19 - TypeScript 5 - Node 24 LTS
Rama	main
Commit	5360bf7 - fix: keep suggestions in sync when the term changes externally
Fecha	24/08/2026
Dominio	terbolinspira.com

Documento generado a partir del código fuente del repositorio. Describe la arquitectura, la estrategia de renderizado y cache, la integración con el CMS, la configuración de seguridad y los procedimientos de despliegue y operación en el servidor de producción.

Indice

1. Introducción	3
Alcance de este documento	3
Documentación relacionada en el repositorio	3
2. Stack tecnológico	3
3. Puesta en marcha	4
Requisitos	4
Scripts disponibles	4
Verificación recomendada antes de publicar	4
4. Variables de entorno	5
Públicas (prefijo NEXT_PUBLIC_)	5
Solo servidor (sin prefijo)	5
Comportamiento ante variables faltantes	5
5. Arquitectura de código	6
Estructura de un feature	6
Reglas de la arquitectura	6
Crear una página nueva	7
Convenciones de nombres	7
6. Mapa de rutas	7
Páginas	7
Route handlers (API interna)	8
Archivos especiales	8
7. Renderizado y cache	8
Páginas con ISR (server-side)	8
Página de productos (CSR)	9
Detalle de producto (SSG + ISR)	9
Tags de revalidación	9
8. Integración con el CMS	10
Contrato general	10
Endpoints consumidos	10
Parámetros del listado de productos	11
9. Imágenes	11

Orígenes permitidos	11
Configuración del optimizador	11
Assets locales bajo subpath	12
10. SEO y metadata	12
11. Seguridad	12
Orígenes permitidos por el CSP	13
Otras medidas	13
12. Soporte de subpath (basePath)	13
13. Infraestructura de producción	14
Datos del entorno	14
Servicios ajenos que conviven en el mismo servidor	15
14. Despliegue	15
Build y empaquetado	15
Deployment ID	16
Procedimiento de actualización	16
Rollback	16
15. Operación y diagnóstico	16
Control del servicio	16
Logs y verificación	17
Problemas frecuentes	17
Cloudflare	17
16. Calidad y pruebas	18
Cobertura actual	18
Observabilidad	18
17. Convenciones de trabajo	18
Reglas operativas	19
18. Estado actual y trabajo pendiente	19
19. Anexo: referencia rápida	19
Rutas y puertos	19
Archivos clave del repositorio	19
Checklist para publicar un cambio	20

1. Introducción

terbol-webapp es el frontend oficial del ecommerce de Terbol. Es una aplicación Next.js 16 con App Router que consume contenido desde un CMS Laravel externo (`cms.terbolinspira.com`) y se publica como un servidor Node sobre Windows Server, detrás de IIS y Cloudflare.

La aplicación **no es un sitio estático**: usa renderizado en servidor (SSR), regeneración incremental (ISR) con revalidación por tags, y expone route handlers propios bajo `/api/*`. Por eso requiere un runtime Node vivo y no puede servirse como archivos planos desde IIS.

Alcance de este documento

- Arquitectura de código y convenciones que debe seguir cualquier desarrollo nuevo.
- Estrategia de renderizado y cache, y como se refresca el contenido del CMS.
- Contrato de integración con el CMS y con el API de productos.
- Configuración de seguridad, imágenes y SEO.
- Infraestructura de producción, procedimiento de despliegue y operación diaria.

Documentación relacionada en el repositorio

Archivo	Contenido
README.md	Guía de entrada: puesta en marcha, scripts y resumen de arquitectura.
docs/arquitectura-cache.md	Detalle de ISR, CSR, Data Cache y el webhook de revalidación.
docs/guia-renderizado-y-estructura.md	Guía extendida de SSG/SSR/ISR/CSR y vertical slices.
docs/products-api-contract.md	Contrato completo del API de productos del CMS.
docs/deploy-windows-iis.md	Instalación desde cero en Windows Server (IIS, ARR, nssm).
docs/operacion-app.md	Operación diaria: actualizar, arrancar, detener, diagnosticar.
docs/plan-cutover-root.md	Plan para mover la app de /qas al dominio principal.
src/features/README.md	Reglas de la arquitectura de features (vertical slices).

2. Stack tecnológico

Capa	Tecnología	Notas
Framework	Next.js 16 (App Router)	React Server Components, ISR, route handlers.
UI	React 19 + React Compiler	reactCompiler activado en next.config.ts.
Lenguaje	TypeScript 5	Verificación con tsc --noEmit.
Estilos	Tailwind CSS v4	Tokens en CSS puro bajo src/styles/.
Animacion	Framer Motion	Componente compartido AnimateOnScroll.

Capa	Tecnología	Notas
Carruseles	Embla Carousel	Con plugin de autoplay.
Correo	Nodemailer	Formulario de contacto (/about).
Sanitización	sanitize-html	HTML proveniente del CMS.
Imágenes	sharp	Optimizador de next/image en el servidor.
Runtime	Node.js 24 LTS	Bloqueado con .nvmrc, .node-versión y engine-strict.
Build	output: standalone	Genera .next/standalone/server.js autocontenido.

NOTA

Los navegadores soportados están declarados en `browserslist` dentro de `package.json`: Chrome/Edge 93+, Firefox 92+, Safari e iOS Safari 15.4+.

3. Puesta en marcha

Requisitos

- **Node.js 24 LTS.** El repositorio fija la versión con `.nvmrc`, `.node-versión` y `engine-strict=true` en `.npmrc`, de modo que instalar con otra versión mayor falla.
- **npm 11** o compatible.

```
nvm install
nvm use

npm ci
cp .env.example .env.local
npm run dev
```

La aplicación queda disponible en `http://localhost:3000`.

Scripts disponibles

Script	Descripción
<code>npm run dev</code>	Servidor de desarrollo en <code>localhost:3000</code> .
<code>npm run build</code>	Build de producción (genera el bundle standalone).
<code>npm run start</code>	Sirve el build de producción.
<code>npm run lint</code>	ESLint con la configuración de <code>eslint-config-next</code> .
<code>npm run test</code>	Suite de tests con <code>node --test</code> y un runner propio.

Verificación recomendada antes de publicar

```
npm run test
npm run lint
./node_modules/.bin/tsc --noEmit
npm run build
```

4. Variables de entorno

Toda la configuración dependiente del entorno se centraliza en `src/config/env.ts`, que es el único punto de verdad. Ningún módulo debe leer `process.env` por su cuenta.

Públicas (prefijo `NEXT_PUBLIC_`)

Variable	Req.	Uso
<code>NEXT_PUBLIC_SITE_URL</code>	Si	URL publica del sitio. Base de metadata SEO, canonicals y Open Graph.
<code>NEXT_PUBLIC_API_URL</code>	Si	URL base del CMS/API para secciones, blog y productos.
<code>NEXT_PUBLIC_STORAGE_URL</code>	Si	URL base de imágenes y archivos servidos por el CMS.
<code>NEXT_PUBLIC_BASE_PATH</code>	No	Subpath opcional del despliegue (ej. <code>/qas</code>). Vacío = la app corre en root.
<code>NEXT_PUBLIC_ASESOR_URL</code>	No	App externa de Venta por Catálogo. Por defecto apunta a PRD.

IMPORTANTE

Las variables `NEXT_PUBLIC_*` se **compilan dentro del build**. Cambiar el dominio, el subpath o la URL de asesores **exige un rebuild**: no alcanza con reiniciar el servicio.

Solo servidor (sin prefijo)

Variable	Req.	Uso
<code>REVALIDATE_SECRET</code>	Si*	Secret del webhook POST <code>/api/revalidate</code> . Viaja en el header <code>x-revalidate-secret</code> .
<code>CMS_REVALIDATE_SECONDS</code>	No	Fallback ISR en segundos. Por defecto 86400 (1 día).
<code>PRODUCTS_API_TOKEN</code>	No	Bearer token si el API de productos lo exige.
<code>PRODUCTS_API_KEY</code>	No	API key si el endpoint usa un header dedicado.
<code>PRODUCTS_API_KEY_HEADER</code>	No	Nombre del header para la API key. Por defecto <code>ApiKey</code> .
<code>SMTP_HOST / SMTP_PORT / SMTP_SECURE</code>	No	Servidor SMTP del formulario de contacto.
<code>SMTP_USER / SMTP_PASS</code>	No	Credenciales del buzón autenticado.
<code>CONTACT_TO / CONTACT_FROM</code>	No	Destinatario y remitente del formulario.

* Requerida para que funcione la revalidación por webhook; sin ella el endpoint responde error controlado.

Comportamiento ante variables faltantes

- **Fail-fast en producción:** si falta NEXT_PUBLIC_SITE_URL, NEXT_PUBLIC_API_URL o NEXT_PUBLIC_STORAGE_URL, el build o el arranque lanzan [env] Falta la variable obligatoria ... en lugar de degradar en silencio a localhost.
- **Degradación controlada:** las variables solo-servidor se leen como opcionales mediante getters lazy. Si falta la configuración SMTP, POST /api/contact responde un 500 controlado y el formulario muestra un error, pero la app no se cae.
- **Nunca** poner secretos en variables NEXT_PUBLIC_*: quedan visibles en el bundle del cliente.

5. Arquitectura de código

El proyecto usa una arquitectura de **vertical slices** (feature-first): el código se organiza por dominio de negocio y no por tipo técnico.

```
src/
  app/           Routing exclusivo (thin pages) + route handlers /api/*
  components/    Componentes compartidos entre features
    layout/      PageLayout, Navbar, Footer, Banner, EndBanner, PageTransition
    ui/          Primitivos: Button, Input, Accordion, AnimateOnScroll
  config/        env.ts, cache.ts, security-headers.ts, image-remote-patterns.ts
  features/      Lógica de negocio por dominio (vertical slices)
    about/ blog/ faq/ home/ products/ promoter/
    science-and-quality/ success-plan/
  lib/           Utilidades transversales (cms-api, seo, base-path, logger, ...)
  styles/        Design tokens en CSS puro (colors, typography, spacing)
```

Estructura de un feature

```
src/features/<nombre>/
  components/    Componentes propios del feature
  views/         Vistas que componen la página completa (<Feature>View.tsx)
  hooks/         (opcional) Hooks propios
  api/           (opcional) Llamadas al CMS / server actions
  data/          (opcional) Normalización de la respuesta del CMS
  types/         (opcional) Tipos del dominio
  index.ts       Barrel export público (solo Views y tipos)
```

Reglas de la arquitectura

Regla	Detalle
Thin pages	Los archivos de app/ solo hacen routing. Son wrappers mínimos que componen un PageLayout y la View del feature. Nunca escribir UI en app/.
Autocontención	Cada dominio encapsula sus componentes, hooks, tipos y vistas dentro de su carpeta.
No cross-imports	Un feature nunca importa del interior de otro feature. Lo compartido se mueve a components/ o lib/.
Barrel exports	Cada feature expone solo lo necesario vía index.ts: como regla general, únicamente Views y tipos.
Tokens de estilo	Colores, tipografía y espacios salen de src/styles/. No hardcodear valores en línea.

Crear una página nueva

Ejemplo para una página de contacto en /contact:

- Crear el feature en src/features/contact/.
- Crear los componentes en components/ con su index.ts.
- Crear la vista views/ContactView.tsx que compone esos componentes.
- Exportarla en el barrel: export { ContactView } from "../views/ContactView";
- Crear src/app/contact/page.tsx como thin wrapper.

```
import { PageLayout } from "@components/layout/PageLayout";
import { ContactView } from "@features/contact";

export default function ContactPage() {
  return (
    <PageLayout>
      <ContactView />
    </PageLayout>
  );
}
```

Convenciones de nombres

Elemento	Convencion	Ejemplo
Carpeta de feature	kebab-case	success-plan/
Vista principal	PascalCase + View	SuccessPlanView
Hero de la página	PascalCase + Hero	SuccessPlanHero
Import desde app/	Desde el barrel	import { XView } from "@features/x"

6. Mapa de rutas

Páginas

Ruta	Feature	Renderizado
/	home	Server Component + ISR
/about	about	Server Component + ISR (incluye formulario de contacto)
/faq	faq	Server Component + ISR
/promoter	promoter	Server Component + ISR
/science-and-quality	science-and-quality	Server Component + ISR
/success-plan	success-plan	Server Component + ISR
/products	products	Client Component (CSR) sobre /api/products

Ruta	Feature	Renderizado
/products/[id]	products	SSG con generateStaticParams + ISR on-demand
/blog	blog	Server Component + ISR
/blog/[id]	blog	Server Component + ISR

El módulo de blog queda accesible por URL directa pero no se publica en la navegación ni en el sitemap mientras este oculto comercialmente.

Route handlers (API interna)

Endpoint	Método	Descripción
/api/products	GET	Listado paginado y filtrado. force-dynamic; responde con Cache-Control: no-store. Ante fallo del CMS devuelve 500 con código PRODUCTS_API_UNAVAILABLE y lista vacía.
/api/products/filters	GET	Catálogos de filtros (tipos de producto, tipos de consumo y enfoques).
/api/revalidate	POST	Webhook del CMS. Valida el header x-revalidate-secret y ejecuta revalidateTag sobre una lista blanca de tags.
/api/contact	POST	Envío del formulario de contacto por SMTP (Nodemailer, runtime nodejs). Incluye honeypot anti-spam y límite de 5000 caracteres por campo.

Archivos especiales

Archivo	Función
app/layout.tsx	Layout raíz: metadataBase, favicons, Google Tag Manager, Google Ads, Navbar, Footer y transiciones de página.
app/sitemap.ts	Sitemap dinámico: rutas estáticas más el detalle de cada producto.
app/robots.ts	robots.txt dinámico; permite todo salvo /api/ y apunta al sitemap.
app/error.tsx	Boundary de error de la aplicación.
app/not-found.tsx	Página 404.

7. Renderizado y cache

La aplicación combina tres patrones según la necesidad de cada página. La elección no es estética: define si el contenido llega en el HTML inicial o si el navegador lo pide después.

Páginas con ISR (server-side)

La mayoría de las páginas (Home, About, Science, Promoter, FAQ, Success Plan) son Server Components que consultan el CMS en el servidor. El HTML llega al navegador con el contenido ya incluido, sin skeletons ni estados de carga.

Mecanismo	Como funciona	Latencia
Webhook	El CMS llama a POST /api/revalidate al publicar un cambio y se purga el tag correspondiente.	Próximo request
Fallback temporal	Si el webhook no se dispara, ISR auto-refresca según CMS_REVALIDATE_SECONDS.	Máximo ~1 día

Página de productos (CSR)

/products usa un componente "use client" que pide los datos con fetch() desde el navegador. Eso habilita filtros, búsqueda y paginacion dinámicos sin recargar la página.

```
Browser -> fetch("/api/products") -> Route Handler -> Data Cache -> CMS
```

El route handler intermedio cachea las respuestas del CMS para no sobrecargar el backend con consultas repetidas, y responde al navegador con Cache-Control: no-store para que el cliente siempre pida datos frescos al servidor.

Detalle de producto (SSG + ISR)

Las páginas /products/[id] se pre-renderizan en el build mediante generateStaticParams. Los productos que no existian al momento del build se renderizan on-demand la primera vez que alguien los visita.

Tags de revalidación

El webhook acepta un tag o un arreglo de tags. Solo se aceptan valores de la lista blanca definida en el route handler:

Tag	Alcance
home	Sección home.
about	Sección about.
success-plan	Sección plan de éxito.
learn	Sección learn (incluye categorías del blog).
help	Sección de ayuda / FAQ.
promoter	Sección promotor.
science	Sección ciencia y calidad.
footer	Contenido global del footer (redes sociales).
advisor-registration	Contenido del registro de asesores.
products	Todo el contenido de productos: listado, filtros y detalle.
blog	Artículos y categorías del blog.
sitemap	Sitemap.

```
POST /api/revalidate
x-revalidate-secret: <REVALIDATE_SECRET>
Content-Type: application/json

{ "tag": "home" }
{ "tag": ["products", "blog"] }
```

ATENCIÓN

Si REVALIDATE_SECRET no coincide entre la app y el CMS, la revalidación falla en silencio desde el punto de vista del editor: el contenido solo se refresca cuando vence el fallback temporal. Al rotar el secret hay que actualizarlo en ambos lados a la vez.

8. Integración con el CMS

Toda la comunicación con el CMS Laravel pasa por `src/lib/cms-api.ts`, que centraliza headers, estrategia de cache y manejo de errores.

Contrato general

- Las respuestas vienen envueltas en un sobre `{ success, message, data }`.
- Los errores HTTP se envuelven en `CmsHttpError`, que conserva el status y el endpoint para facilitar el diagnóstico; existe el helper `isCmsNotFoundError` para distinguir un 404 legítimo.
- Cada fallo se registra con `logError("cms_fetch_failed", ...)` y se relanza para que lo tome el boundary de error correspondiente.
- El HTML que llega del CMS se sanitiza con `src/lib/html-sanitizer.ts` antes de renderizarse.

Endpoints consumidos

Endpoint del CMS	Tags	Uso
<code>/sections/home</code>	home	Contenido de la home.
<code>/sections/about</code>	about	Contenido de about.
<code>/sections/success-plan</code>	success-plan	Plan de éxito.
<code>/sections/learn</code>	learn, blog	Learn y categorías del blog.
<code>/sections/help</code>	help	FAQ / ayuda.
<code>/sections/promoter</code>	promoter	Promotor.
<code>/sections/science</code>	science	Ciencia y calidad.
<code>/footer</code>	footer	Redes sociales del footer.
<code>/advisor-registration</code>	advisor-registration	Registro de asesores.
<code>/products</code>	products	Listado de productos.
<code>/products/{id}</code>	products	Detalle de producto.
<code>/products/types</code>	products	Catálogo de tipos de producto.

Endpoint del CMS	Tags	Uso
/products/consumption-types	products	Catálogo de tipos de consumo.
/products/focuses	products	Catálogo de enfoques.

Parámetros del listado de productos

Parámetro	Tipo	Descripción
page	number	Página actual. Mínimo 1.
limit	number	Cantidad por página. La app limita a un máximo de 60.
product_type_ids[]	string[]	IDs de tipo de producto del CMS.
consumption_type_ids[]	string[]	IDs de tipo de consumo del CMS.
focus_ids[]	string[]	IDs de enfoque del CMS.
name	string	Búsqueda libre por nombre. Solo se envía si tiene valor.

NOTA

En producción no hay datos locales de respaldo: si el CMS no responde, la sección queda vacía o muestra un error controlado. Los mocks locales existen solo para desarrollo y tests. El contrato completo está en docs/products-api-contract.md.

9. Imágenes

Las imágenes se sirven con next/image. Los orígenes remotos permitidos se construyen en src/config/image-remote-patterns.ts, que combina una lista fija con los hosts derivados de NEXT_PUBLIC_STORAGE_URL y NEXT_PUBLIC_API_URL.

Orígenes permitidos

Host	Motivo
cms.terbolinspira.com	Imágenes y archivos del CMS.
terbol.blob.core.windows.net	Almacenamiento en Azure Blob.
images.unsplash.com	Imágenes de ejemplo.
i.ytimg.com	Miniaturas del facade de YouTube (carga diferida del player).
localhost / 127.0.0.1 / 192.168.0.7	Desarrollo local.

Configuración del optimizador

Opción	Valor
formats	image/avif, image/webp

Opción	Valor
deviceSizes	640, 750, 828, 1080, 1200
imageSizes	16, 32, 48, 64, 96, 128, 256, 384
minimumCacheTTL	86400 segundos (1 día)
unoptimized	Activado automáticamente solo cuando hay basePath

IMPORTANTE

Bajo un basePath el optimizador de Next no antepone el prefijo al fetch interno de las imágenes locales y estas devuelven un 400. Por eso, cuando hay subpath, la configuración fuerza `unoptimized: true`: Next aplica el basePath al src directo y las imágenes cargan. En root la optimización queda intacta.

Assets locales bajo subpath

`next/image` no aplica el basePath al src de imágenes locales de `/public`, ni siquiera con `unoptimized`. Por eso esos src se construyen con el helper `assetPath()` de `src/lib/base-path.ts`.

10. SEO y metadata

- **metadataBase** se arma en el layout raíz a partir de `env.SITE_URL`, lo que permite a Next construir URLs absolutas para Open Graph, Twitter cards e imágenes.
- **Canonicals** y URLs absolutas se generan con los helpers `getCanonicalPath()` y `getAbsoluteUrl()` de `src/lib/seo.ts`.
- **createPageMetadata()** centraliza título, descripción, canonical, Open Graph e indexación de cada página, con imágenes por defecto declaradas en `SEO_IMAGES`.
- **Sitemap** dinámico con rutas estáticas más el detalle de cada producto obtenido del CMS.
- **robots.txt** dinámico: permite todo, bloquea `/api/` y publica la URL del sitemap según el entorno.

NOTA

Como todas las URLs derivan de `NEXT_PUBLIC_SITE_URL`, cambiar el dominio o el subpath solo requiere ajustar esa variable y rebuildear. No hay dominios hardcodeados en el código de SEO.

11. Seguridad

Las cabeceras se definen en `src/config/security-headers.ts` y se aplican a todas las rutas desde `next.config.ts`. Hay tests que verifican su contenido.

Cabecera	Valor
Content-Security-Policy	Política por directiva: <code>default-src 'self', object-src 'none', frame-ancestors 'none', base-uri 'self', form-action 'self'</code> , y listas explícitas para <code>script</code> , <code>img</code> , <code>connect</code> , <code>media</code> y <code>frame</code> .
Strict-Transport-Security	<code>max-age=63072000; includeSubDomains; preload</code>
X-Frame-Options	DENY

Cabecera	Valor
X-Content-Type-Options	nosniff
Referrer-Policy	strict-origin-when-cross-origin
Permissions-Policy	camera, microphone, geolocation, payment, usb, serial, bluetooth, magnetometer e interest-cohort deshabilitados
X-DNS-Prefetch-Control	on
X-Download-Options	noopen
X-Permitted-Cross-Domain-Policies	none

Orígenes permitidos por el CSP

Directiva	Orígenes
script-src	'self', 'unsafe-inline', Google Tag Manager, Google Ads, DoubleClick. En desarrollo se agrega 'unsafe-eval'.
connect-src	'self', CMS, Google Analytics/Tag Manager/DoubleClick y hosts locales. En desarrollo se agregan ws: y wss:.
img-src	'self', data:, blob:, CMS, Unsplash, miniaturas de YouTube, pixeles de Google.
frame-src	'self', YouTube (incluido youtube-nocookie), Vimeo, Google Tag Manager, DoubleClick.
media-src	'self', data:, blob:, CMS.
style-src / font-src	'self' con 'unsafe-inline' para estilos, y data: para fuentes.

Otras medidas

- **poweredByHeader** deshabilitado: no se expone la cabecera X-Powered-By.
- **Sanitización de HTML** del CMS antes de renderizarlo, con tests dedicados.
- **Honeypot anti-spam** y límite de longitud por campo en el formulario de contacto.
- **Secretos solo en el servidor**: el objeto `serverEnv` se importa únicamente desde route handlers y server components, nunca desde código cliente.
- **El proceso Node escucha solo en 127.0.0.1**; el tráfico externo entra por IIS y Cloudflare.

12. Soporte de subpath (basePath)

La aplicación puede montarse bajo una ruta en lugar de la raíz del dominio. Todo el mecanismo depende de una única variable, `NEXT_PUBLIC_BASE_PATH`.

Pieza	Responsabilidad
<code>next.config.ts</code>	Define <code>basePath</code> cuando la variable existe y activa <code>images.unoptimized</code> .
<code>src/lib/base-path.ts</code>	Exporta <code>BASE_PATH</code> y los helpers <code>apiPath()</code> y <code>assetPath()</code> .

Pieza	Responsabilidad
<code>apiPath()</code>	Antepone el prefijo a los <code>fetch()</code> manuales hacia route handlers propios. Next no lo hace automáticamente.
<code>assetPath()</code>	Antepone el prefijo al <code>src</code> de imágenes locales de <code>/public</code> .
<code>src/lib/image-assets.ts</code>	Aplica <code>BASE_PATH</code> a los assets locales al resolver imágenes del CMS con fallback local.

IMPORTANTE

Next aplica el `basePath` automáticamente a `<Link>`, a `next/image` y a los assets del build, pero **no** a los `fetch()` manuales ni al `src` de imágenes locales. Esos dos casos son los que cubren los helpers.

Cuando la variable no está definida, `BASE_PATH` queda vacío y los helpers devuelven la ruta sin cambios: el mismo código sirve para `root` y para `subpath` sin condicionales dispersos.

13. Infraestructura de producción

La app se publica como un servidor Node sobre Windows Server, con IIS haciendo reverse proxy y Cloudflare terminando el HTTPS.

```
Cloudflare (termina HTTPS, modo SSL Flexible)
  | https://terbolinspira.com (y www)
  v HTTP:80 al origen
IIS (ARR + URL Rewrite, reverse proxy vía web.config)
  v
node server.js en 127.0.0.1:3001 (build standalone de Next)
  ^
  |__ lo mantiene vivo el servicio de Windows "TerbolWeb" (nssm)
```

Datos del entorno

Componente	Valor
Carpeta de la app	C:\Terbol\webapp
Bundle ejecutado	C:\Terbol\webapp\next\standalone\server.js
Puerto interno	3001, solo 127.0.0.1
Servicio de Windows	TerbolWeb, administrado con nssm (C:\Tools\nssm.exe)
Node	C:\Users\Administrator\AppData\Local\nvm\v24.16.0\node.exe
Variables	C:\Terbol\webapp\.env.production
Logs	C:\Terbol\webapp\logs\out.log y err.log
Sitio IIS (proxy)	C:\Terbol\webapp-proxy\web.config
CMS	Laravel en cms.terbolinspira.com

NOTA

El servicio apunta a la **ruta versionada real** de Node y no al symlink de nvm (C:\nvm4w\nodejs), para que un nvm use no rompa el arranque.

Servicios ajenos que conviven en el mismo servidor

El servidor aloja otros sistemas que **no** deben tocarse al operar esta aplicación:

Sistema	Detalle
Landing Astro	Sitio estático en Default Web Site (C:\inetpub\wwwroot), con binding catch-all.
CMS Laravel	cms.terbolinspira.com, en C:\inetpub\wwwroot\cms.
Laragon	Apache en el puerto 81, MySQL 3306, MailHog 1025/8025.
Aplicación .NET	C:\App.
Jobs de SQL Server	C:\Terbol\TerbolInspira. No confundir con C:\Terbol\webapp.

ATENCIÓN

Nunca ejecutar iisreset: reinicia todos los sitios del servidor, incluido el CMS. Usar siempre Stop-Website / Start-Website con el nombre del sitio propio.

14. Despliegue

Build y empaquetado

El build usa output: "standalone", que produce un server.js autocontenido en .next/standalone.

```
cd C:\Terbol\webapp
npm ci                                # solo si cambio package.json / lock
$deployId = git rev-parse --short HEAD
$env:NEXT_DEPLOYMENT_ID = $deployId
npm run build
```

Completar el bundle standalone (paso crítico)

Next **no incluye** los assets del cliente (.next/static) ni public/ en el standalone, porque asume un CDN. Sin CDN hay que copiarlos para que server.js pueda servirlos.

```
$std = "C:\Terbol\webapp\.next\standalone"
Remove-Item -Recurse -Force "$std\.next\static" -ErrorAction SilentlyContinue
Remove-Item -Recurse -Force "$std\public" -ErrorAction SilentlyContinue
New-Item -ItemType Directory -Force "$std\.next\static" | Out-Null
New-Item -ItemType Directory -Force "$std\public" | Out-Null
Copy-Item -Recurse -Force "C:\Terbol\webapp\.next\static\*" "$std\.next\static"
Copy-Item -Recurse -Force "C:\Terbol\webapp\public\*" "$std\public"
Copy-Item -Force "C:\Terbol\webapp\.env.production" "$std\.env.production"
```


ATENCIÓN

Bug conocido del Copy-Item. Si el destino ya existe, `Copy-Item -Recurse origen destino` copia la carpeta *adentro* y genera `...\static\static\...`. Eso produce 404 en `/_next/static/*` y el sitio se ve **sin estilos**. Por eso siempre hay que borrar el destino y copiar el **contenido** con `*`.

Deployment ID

Antes del build se define `NEXT_DEPLOYMENT_ID` con el SHA corto del commit. Next lo inserta en el HTML como `data-dpl-id`, agrega `?dpl=<id>` a los assets estáticos y detecta si un cliente intenta navegar con una versión anterior del build; ante un desajuste fuerza una navegación completa en lugar de reutilizar una navegación client-side vieja. Es lo que evita que, tras un deploy, se mezclen assets o payloads de builds distintos.

Procedimiento de actualizacion

```
cd C:\Terbol\webapp

git pull
git diff HEAD@{1} --name-only | findstr package      # si lista package*, npm ci

C:\Tools\nssm.exe stop TerbolWeb                    # libera el lock de standalone
$env:NEXT_DEPLOYMENT_ID = git rev-parse --short HEAD
npm run build

# ... completar el bundle standalone (bloque anterior) ...

C:\Tools\nssm.exe start TerbolWeb
C:\Tools\nssm.exe status TerbolWeb                  # SERVICE_RUNNING
curl.exe -s -o NUL -w "%{http_code}" http://localhost:3001
```

IMPORTANTE

El servicio **debe** estar detenido durante el build: el proceso Node corre desde `.next\standalone` y bloquea la carpeta. Si no se para, `next build` falla con `EBUSY: resource busy or locked`.

Si solo cambiaron variables **sin** el prefijo `NEXT_PUBLIC_`, no hace falta rebuild: alcanza con copiar el `.env.production` al standalone y reiniciar el servicio.

Rollback

Detener el sitio IIS devuelve el tráfico del dominio al landing estático anterior, que conserva el binding catch-all. La aplicación y su servicio quedan intactos para reintentar.

```
Stop-Website -Name "TerbolWeb"      # vuelve el landing de respaldo
Start-Website -Name "TerbolWeb"     # vuelve la aplicación
```

15. Operación y diagnóstico

Control del servicio

```
C:\Tools\nssm.exe start TerbolWeb
C:\Tools\nssm.exe stop TerbolWeb
C:\Tools\nssm.exe restart TerbolWeb
C:\Tools\nssm.exe status TerbolWeb # SERVICE_RUNNING | SERVICE_STOPPED
```

El servicio arranca automáticamente al encender el servidor, así que no hay que hacer nada tras un reinicio.

Logs y verificación

```
Get-Content C:\Terbol\webapp\logs\err.log -Tail 30
Get-Content C:\Terbol\webapp\logs\out.log -Tail 30
Get-EventLog -LogName Application -Source nssm -Newest 5 | Format-List
netstat -ano | findstr ":3001"
```

Para aislar en que capa falla un asset, conviene recorrer la cadena completa:

```
$asset = "/_next/static/chunks/<hash>.css"

curl.exe -s -o NUL -w "node: %{http_code}" "http://localhost:3001$asset"
curl.exe -s -o NUL -w "iis: %{http_code}" -H "Host: terbolinspira.com" `
    "http://localhost$asset"
curl.exe -s -o NUL -w "cf: %{http_code}" "https://terbolinspira.com$asset"
```

Problemas frecuentes

Síntoma	Causa	Solución
EBUSY rmdir .next\standalone durante el build	El servicio Node bloquea la carpeta	Detener el servicio antes de npm run build
El sitio carga sin estilos, 404 en /_next/static/*	Copy-Item anido la carpeta (static\static)	Rehacer la copia con remove más * y reiniciar
SERVICE_STOPPED con err.log vacío	Ruta de Node incorrecta (cambio por nvm)	Apuntar a la ruta versionada real de nvm
Puerto 3001 ocupado	Quedo un node server.js manual corriendo	netstat para hallar el PID y taskkill /F
Build falla con [env] Falta la variable...	Falta una NEXT_PUBLIC_* requerida	Completar .env.production y rebuildear
Imágenes del CMS no cargan	Host no permitido en remotePatterns	Verificar NEXT_PUBLIC_STORAGE_URL
node responde 200 pero Cloudflare 404	Cloudflare sirve cache vieja	Caching > Purge Everything
Contenido del CMS no se refresca	Webhook mal configurado o secret distinto	Revisar URL y x-revalidate-secret en ambos lados

Cloudflare

- Termina el HTTPS. El modo SSL debe permanecer en **Flexible**: el origen IIS escucha en HTTP:80 y no tiene certificado en el 443. Subirlo a Full rompería el sitio.
- cf-cache-status: DYNAMIC en el HTML indica que no se cachea la página, que es lo esperado.

- Tras un cambio grande o un cutover, ejecutar **Purge Everything**.

16. Calidad y pruebas

La suite corre con `node --test` sobre un runner propio (`test/test-runner.cjs`) que compila TypeScript al vuelo y mockea `next/cache`, de modo que no hace falta un framework externo.

Cobertura actual

Area	Archivos de test
Route handlers	api/contact, api/products, api/products/filters, api/revalidate
SEO	sitemap.test.ts, lib/seo.test.ts
Configuración	security-headers.test.ts, image-remote-patterns.test.ts
Datos del CMS	cms-data, cms-page-data, cmsHome, cmsBlog
Productos	products-api.test.ts, targetAudiences.test.ts
Utilidades	html-sanitizer, image-assets, logger

En conjunto cubren los contratos críticos: revalidación, productos, normalización del CMS, sanitización de HTML y cabeceras de seguridad.

Observabilidad

`src/lib/logger.ts` expone `logInfo`, `logWarn` y `logError` con contexto estructurado por request (`getRequestLogContext`) y medición de duración (`getDurationMs`). Los route handlers registran resultado, cantidad de items, origen de los datos y tiempo de respuesta.

17. Convenciones de trabajo

El proyecto sigue **Conventional Commits**, con commits lógicos y atómicos en lugar de un macro-commit por tarea.

```
<tipo>(<ámbito opcional>): <descripción corta>
```

Tipo	Uso
feat	Nueva característica o funcionalidad.
fix	Correccion de un error.
refactor	Cambio de código que no agrega features ni corrige bugs.
style	Cambios de formato sin impacto en la lógica.
docs	Cambios en documentación.
chore	Mantenimiento, configuración, dependencias.
test	Agregar o modificar pruebas.

Reglas operativas

- No commitear `.env.production` ni ningún archivo con secretos; `.gitignore` ya excluye todos los `.env*` salvo `.env.example`.
- No ejecutar `npm audit fix --force`: introduce cambios rompieses.
- Mantener `.env.example` actualizado cuando se agrega una variable nueva.

18. Estado actual y trabajo pendiente

Al momento de generar este documento, la aplicación se publica bajo el subpath `/qas` del dominio principal, mientras que la raíz sigue servida por el landing Astro anterior.

Aspecto	Estado actual	Estado objetivo
Webapp	terbolinspira.com/qas	terbolinspira.com y www
Landing Astro	Sirve la raíz del dominio	Respaldo: entra si se detiene el sitio de la app
/qas	Ambiente publicado	Redirect 301 permanente hacia la ruta equivalente
Links de asesores	/VentaPorCatalogo/QAS	/VentaPorCatalogo/PRD
Optimizador de imágenes	Desactivado por el basePath	Activo (requiere sharp en el standalone)

El procedimiento completo de esa migración, con fases, verificaciones y plan de rollback, está documentado en `docs/plan-cutover-root.md`.

IMPORTANTE

La documentación de despliegue heredada describe un escenario de raíz que todavía no se aplicó. Al completar el cutover hay que actualizar `docs/deploy-windows-iis.md` y `docs/operacion-app.md` para que reflejen el estado real.

19. Anexo: referencia rápida

Rutas y puertos

Concepto	Valor
Desarrollo local	<code>http://localhost:3000</code>
Proceso Node en producción	<code>127.0.0.1:3001</code>
Dominio público	<code>terbolinspira.com</code> y <code>www.terbolinspira.com</code>
CMS	<code>https://cms.terbolinspira.com/api</code> y <code>/storage</code>
Webhook de revalidación	POST <code>/api/revalidate</code> con header <code>x-revalidate-secret</code>

Archivos clave del repositorio

Archivo	Responsabilidad
next.config.ts	basePath, deploymentId, output standalone, cabeceras, configuración de imágenes.
src/config/env.ts	Único punto de verdad de la configuración por entorno, con fail-fast.
src/config/cache.ts	Fallback temporal de revalidación ISR.
src/config/security-headers.ts	CSP y demas cabeceras de seguridad.
src/config/image-remote-patterns.ts	Hosts permitidos para next/image.
src/lib/cms-api.ts	Cliente del CMS con tags de cache y manejo de errores.
src/lib/base-path.ts	BASE_PATH y helpers apiPath() / assetPath().
src/lib/seo.ts	Canonicals, URLs absolutas y metadata por página.
src/lib/html-sanitizer.ts	Sanitización del HTML que llega del CMS.
src/lib/logger.ts	Logging estructurado con contexto de request.

Checklist para publicar un cambio

- Correr `npm run test`, `npm run lint`, `tsc --noEmit` y `npm run build` en local.
- Verificar si el cambio toca alguna variable `NEXT_PUBLIC_*`: si es así, hace falta rebuild en el servidor.
- Detener el servicio antes de buildear en producción.
- Completar el bundle standalone con el bloque de copia correcto (remove más `\`).
- Verificar `localhost:3001` antes de exponer el cambio.
- Confirmar el `data-dpl-id` del HTML contra el SHA del commit desplegado.
- Purgar Cloudflare si el cambio afecta assets o HTML cacheado.